

WINE

Техническая документация frontend · Kotlin Multiplatform +
Compose Multiplatform Android/iOS

WINE

Техническая документация frontend

Kotlin Multiplatform + Compose Multiplatform Android / iOS

Назначение документа

Документ описывает фактическую frontend-реализацию по предоставленному архиву проекта: архитектуру, исходную структуру, UI, навигацию, состояние, сетевой слой, локальное хранение, on-device обработку изображений, платформенные различия, сборку и эксплуатационные замечания. Документ предназначен для разработчика, которому необходимо быстро понять кодовую базу и продолжить её развитие.

Параметр	Значение
Проект	WineScanner
Модель приложения	Kotlin Multiplatform / Compose Multiplatform
Платформы	Android; iOS
Основной UI	Compose Multiplatform + Material 3
Навигация	Voyager TabNavigator / ScreenModel
Сеть	Ktor Client + Kotlinx Serialization
Локальные данные	SQLDelight + Multiplatform Settings
On-device ML	ONNX Runtime на Android; iOS-инференс не подключён

Содержание

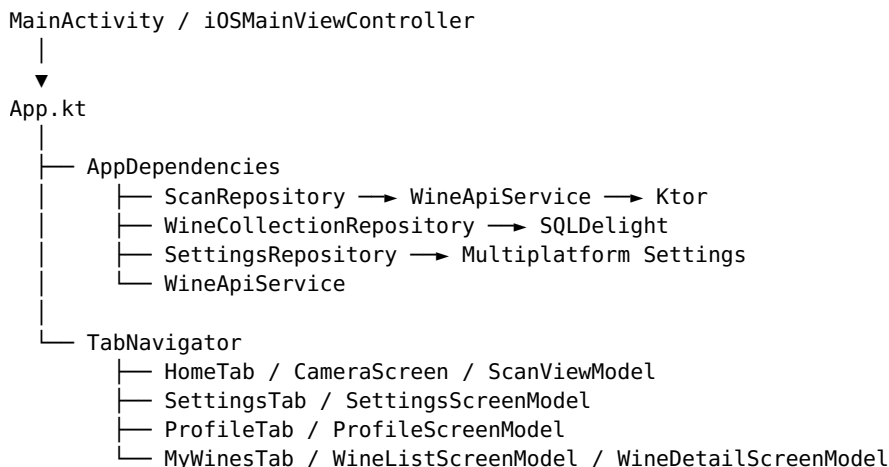
№	Раздел
1	Резюме архитектуры
2	Структура проекта
3	Стек и зависимости
4	Точка входа и сборка приложения
5	Архитектура и зависимости
6	Навигация и жизненный цикл экранов
7	Сценарий сканирования вина
8	Сетевой слой и API-контракт
9	Локальное хранение и кэш
10	UI-слой и дизайн-система
11	Локализация и настройки

- 12 Платформенные реализации Android / iOS
 - 13 Ошибки, loading и устойчивость
 - 14 Сборка, конфигурация и окружения
 - 15 Тестирование и качество
 - 16 Технический долг и рекомендации
 - 17 Onboarding нового frontend-разработчика
 - 18 Справочник файлов
-

1. Резюме архитектуры

Frontend представляет собой кроссплатформенное приложение на Kotlin Multiplatform. Бизнес-логика, модели, сетевой слой, локальное хранилище и большая часть Compose UI находятся в commonMain. Платформенные различия вынесены в androidMain и iosMain через expect/actual. Главная архитектурная идея — держать UI и доменные правила общими, а доступ к камере, SQLite-драйверу, настройкам, HTTP engine и image/ML APIs — платформенным.

Ключевая схема:



Основные свойства решения

- UI построен декларативно на Compose Multiplatform и Material 3.
- Состояние экранов хранится в StateFlow внутри Voyager ScreenModel либо в локальном ScanViewModel.
- Сетевые DTO отделены от доменной модели WineResult.
- Локальная коллекция является источником данных для «Мои вина» и достижений.
- Изображение вина может храниться в SQLite как Base64-текст.
- Android использует CameraX и ONNX Runtime; iOS использует AVFoundation, но on-device ONNX-инференс ещё не подключён.

Архитектурный вывод

Кодовая база уже разделяет UI, domain, network, persistence и platform adapters. Главная зона роста — формализовать DI/создание зависимостей, унифицировать обработку ошибок и завершить платформенные функции iOS. При расширении функциональности рекомендуется сохранять существующее направление зависимостей: UI → ScreenModel/ViewModel → Repository/Service → platform implementation.

2. Структура проекта

Путь	Назначение
composeApp/src/commonMain/kotlin/com/winescanner/app	Общий Kotlin-приложения
commonMain/.../presentation	Compose-экраны и модели состояния экранов
commonMain/.../domain	Репозитории, доменные модели, достижения
commonMain/.../model	Сетевые/общие модели данных ScanUiState
commonMain/.../network	Ktor client, AF service, JSON-контракт
commonMain/.../navigation	AppDependencies нижняя навигация
commonMain/.../ui	Локализация тема
commonMain/.../camera	expect CameraControll геометрия gu frame
commonMain/.../ml	expect LabelExtractor
commonMain/.../db	expect фабрик SQLDelight и Settings
commonMain/sqlDelight	SQL-схема и queries
commonMain/composeResources	Изображения достижений
androidMain	Android Camera ONNX Runtime OkHttp, SQLite driver, SharedPreferences
iosMain	AVFoundation, Darwin engine NativeSqliteDelegate NSUserDefaults



Рекомендуемая ментальная модель каталогов

presentation/ → что пользователь видит и как экран реагирует
domain/ → что приложение делает с данными
network/ → как данные приходят с backend
model/ → DTO и UI-state контракты
db/ → платформенный доступ к локальной БД/настройкам
camera/ml/ → платформенные медиа/ML адаптеры
ui/ → тема, строки и визуальные примитивы

3. Стек и зависимости

Технология	Версия из version catalog	Роль
Kotlin	2.4.0	Язык и KMP
Android Gradle Plugin	9.0.0	Android build
Compose Multiplatform	1.11.1	Общий UI
Ktor	3.4.0	HTTP client
Kotlin Coroutines	1.10.1	Асинхронность / Flow
Kotlinx Serialization	1.8.1	JSON
AndroidX Activity Compose	1.10.1	Android Compose integration
CameraX	1.5.1	Android camera
ONNX Runtime Android	1.27.0	Локальная детекция этикетки
Voyager	1.1.0-beta03	Navigation / ScreenModel
SQLDelight	2.3.2	SQLite abstraction
Multiplatform Settings	1.3.0	Настройки приложения

Принцип выбора зависимостей

Ключевые инфраструктурные зависимости вынесены в `libs.versions.toml`, поэтому версии централизованы. Это упрощает синхронизацию KMP-модулей и обновления.

4. Точка входа и сборка приложения

Android

AndroidManifest объявляет CAMERA и INTERNET permissions, требует наличие камеры и запускает MainActivity в portrait orientation. MainActivity создаёт три платформенные зависимости — LabelExtractor, SqlDriverFactory и SettingsFactory — и передаёт их в App(). Там создаются database, HttpClient, WineApiService и репозитории.

iOS

iOS имеет MainViewController, платформенные actual-реализации камеры, HTTP engine, SQLDelight driver и Settings. iOS targets в Gradle — iosArm64 и iosSimulatorArm64. iosX64 сознательно не включён.

Android build configuration

Параметр	Значение
namespace	com.winescanner.app
applicationId	com.winescanner.app
compileSdk	36
minSdk	24

targetSdk	36
Java/Kotlin JVM target	17
versionCode / versionName	1 / 1.0

5. Архитектура и зависимости

Composition root

App.kt выступает composition root. Здесь один раз создаются WineDatabase, HttpClient, WineApiService и репозитории. Затем AppDependencies передаётся через LocalAppDependencies. Это простой ручной DI-механизм без отдельного DI-фреймворка.

```
MainActivity.onCreate()  
├─ LabelExtractor(applicationContext)  
├─ SqlDriverFactory(applicationContext)  
├─ SettingsFactory(applicationContext)  
└─ setContent { App(...) }
```

Компонент	Зависит от	Ответственно
ScanViewModel	ScanRepository, WineCollectionRepository	Оркестрация s flow и UI state
ScanRepository	LabelExtractor, WineApiService	Подготовка фо вызов API
WineApiService	HttpClient	HTTP + JSON + mapping
WineCollectionRepository	WineDatabase	CRUD коллекц и mapping SQL domain
AchievementEngine	SavedWine	Локальный рас достижений
SettingsRepository	Settings	Имя, аватар, яз звук
WineDetailScreenModel	WineCollectionRepository, WineApiService	cache-first deta refresh

Граница слоёв

Важное архитектурное решение — доменная SavedWine не равна SQLDelight SavedWineEntity и не равна сетевой WineResult. Это снижает связность: изменение JSON или SQLite-схемы должно затрагивать в первую очередь соответствующий mapper.

6. Навигация и жизненный цикл экранов

Основная навигация построена через Voyager TabNavigator. HomeTab является стартовой вкладкой. Нижний бар реализован собственным Compose-компонентом WineBottomBar и содержит четыре Tab: HomeTab, SettingsTab, ProfileTab, MyWinesTab.

Вкладка	Основной UI	Состояние
Главная	HomeTab → CameraScreen / Loading / Result	ScanViewModel

Настройки	SettingsTab	SettingsScreenModel
Профиль	ProfileTab	ProfileScreenModel + SettingsRepository
Мои вина	MyWinesTab → detail	WineListScreenModel / WineDetailScreenModel
Лента	FeedTab	FeedViewModel

State-driven UI

Для локальных экранов используется ScreenModel из Voyager и screenModelScope. Это позволяет подписываться на Flow БД и выполнять suspend-операции без ручного управления CoroutineScope на каждом Composable.

```
ScanUiState
    Camera(errorMessage?)
    Loading
    Result(wine)
```

UI не хранит бизнес-состояние распознавания отдельно: экран отображает состояние, которое публикует ScanViewModel.

7. Сценарий сканирования вина

Полный пользовательский поток:

1. CameraScreen показывает CameraPreview + GuideFrame
2. Пользователь нажимает ShutterButton
3. CameraController.capturePhoto() → JPEG ByteArray
4. ScanViewModel.onPhotoCaptured(photo)
5. state = Loading
6. ScanRepository.identifyWine(photo)
7. LabelExtractor.cropToGuideFrame(photo)
8. WineApiService.identifyWine(crop)
9. Base64(JPEG) → POST /api/recognize
10. Response → WineResult
11. WineCollectionRepository.save(...)
12. state = Result(wine)
13. ResultScreen показывает карточку

Локальная обработка изображения

В текущем ScanRepository используется именно cropToGuideFrame(), то есть перед отправкой берётся центральная область, соответствующая UI-рамке. ONNX-детектор не вызывается из текущего scan flow. Android LabelExtractor всё же содержит реализацию extractLabelCrop() с ONNX Runtime и fallback; iOS содержит только геометрический crop.

GuideFrame

Константа	Значение
WIDTH_FRACTION	0.72
HEIGHT_FRACTION	0.48
Источник геометрии	GuideFrame.bounds
Назначение	Единые границы UI-рамки и crop

8. Сетевой слой и API-контракт

HTTP stack

WineApiService использует Ktor HttpClient. ContentNegotiation настроен на kotlinx.serialization JSON, unknown keys игнорируются, JSON допускает lenient parsing. Платформенный engine выбирается через expect/actual: OkHttp на Android и Darwin на iOS.

Endpoint	Метод	Назначение
/api/recognize	POST	Распознать вино по Base64 JPEG сгор
/api/wines/{slug}	GET	Получить детали конкретного вина
/feed	GET	Получить информацию для ленты

Контракт распознавания

POST /api/recognize
Content-Type: application/json

```
{
  "image_base64": "<JPEG as Base64>"
}
```

response:

```
{
  "status": "...",
  "url": "...",
  "parsed_data": {
    "name": "...", "description": "...", "factory": "...",
    "rate": "...", "area": "...", "sort": "...",
    "type": "...", "color": "...", "temperature": "...",
    "alcohol": "...", "dishes": [...], "image_base64": "..."
  }
}
```

Mapping

Сервер не присылает отдельный slug. WineApiService извлекает последний segment из url. Если url отсутствует или пуст, сервис бросает WineApiException, потому что wineSlug нужен как primary key локальной коллекции.

```
WineDetailScreenModel
├─ getBySlug() → сразу показать локальный SavedWine
└─ refreshFromServer()
   ├─ GET /api/wines/{slug}
   ├─ если успех → INSERT OR REPLACE
   └─ если ошибка → оставить локальный cache
```

9. Локальное хранение и кэш

SQLDelight

WineDatabase.sq описывает таблицу SavedWineEntity. wineSlug — TEXT PRIMARY KEY. Коллекция сортируется по scannedAtEpochMillis DESC. Есть selectAll, selectBySlug, selectByWinery, countAll, insertOrReplace и deleteBySlug.

Поле	Тип	Назначение
------	-----	------------

wineSlug	TEXT PK	Идентификатор вина из URL
wineName	TEXT	Название
description	TEXT	Описание
winery	TEXT?	Производитель
rating	TEXT?	Рейтинг, включая «Нет рейтинга»
area / grapeVariety / type / color	TEXT?	Атрибуты вина
temperature / alcohol	TEXT?	Подача и крепость
dishes	TEXT	Список блюд через разделитель \ \ \
imageBase64	TEXT?	Изображение
scannedAtEpochMillis	INTEGER	Дата первого/последнего сохранённого скана

Cache-first detail

Это важный UX-паттерн: наличие сохранённого вина не зависит от доступности backend. При этом актуальные данные могут быть обновлены при открытии карточки.

Настройки

Multiplatform Settings хранит имя профиля, Base64 аватара, язык и громкость, а также настройки темы.

10. UI-слой и дизайн-система

Compose

Основные экраны реализованы как @Composable функции. Общая тема WineScannerTheme использует MaterialTheme с darkColorScheme/LightColorScheme.

Переиспользуемые UI primitives

- GlassCard — единый контейнер карточки с градиентом и border; это визуальная имитация glassmorphism без backdrop blur.
 - ScreenHeader — общий заголовок экранов с символом бокала.
 - WineBottomBar — единый нижний tab bar с pressed/active animation.
 - GuideFrame — единый источник размеров рамки камеры.
-

11. Локализация и настройки

Локализация

Строки не хранятся в Android strings.xml как основной механизм. В commonMain определён интерфейс AppStrings и три реализации: RussianStrings, EnglishStrings и ChineseStrings. LocalStrings — staticCompositionLocalOf, а App выбирает реализацию по languageCode.

Код	Язык
ru	Русский
en	English
zh	Китайский

Поток смены языка

```

SettingsTab
→ SettingsScreenModel.onLanguageSelected()
→ SettingsRepository.setLanguageCode()
→ _languageCode.value = code
→ App.collectAsState()
→ выбирается новый AppStrings
→ Compose recomposition

```

Это даёт мгновенную смену UI-строк без отдельного navigation reset. При этом AndroidManifest/ресурсы платформы остаются отдельной системой и не заменяют общий AppStrings.

Звук

Настройка soundVolume хранится как Float в диапазоне 0..1.

12. Платформенные реализации Android / iOS

Android

- CameraX: Preview, ImageCapture, runtime permission.
- Ktor OkHttp engine.
- SQLDelight Android driver.
- SharedPreferencesSettings.
- ONNX Runtime 1.27.0 для локального LabelExtractor.
- Debug crop сохраняется в app-specific external storage при `DEBUG_SAVE_CROPS = true`.
- MainActivity скрывает navigation bar и использует edge-to-edge.

Android ONNX

Модель лежит в `composeApp/src/androidMain/assets` как `label_detector.onnx` и `label_detector.onnx.data`; суммарный размер около 24 МБ. В текущем коде `extractLabelCrop` ожидает output формы `[1, 5] = x1, y1, x2, y2, confidence`. Однако текущий `ScanRepository` отправляет `cropToGuideFrame()`, поэтому этот ONNX path не является обязательным этапом текущего сценария.

iOS

- AVFoundation для камеры.
- Ktor Darwin engine.
- NativeSqliteDriver.
- NSUserDefaultsSettings.
- Image decoding через Skia.
- `LabelExtractor.extractLabelCrop` содержит TODO: подключить ONNX Runtime через CocoaPod `onnxruntime-objc`.
- `AvatarPicker.pickAvatar()` пока возвращает null; TODO — `PHPickerViewController`.

Платформенный риск

Функциональное поведение Android и iOS не полностью симметрично: Android имеет рабочий on-device ONNX adapter, а iOS — fallback/геометрический crop.

13. Ошибки, loading и устойчивость

Распознавание

Событие	UI-реакция
Фото успешно получено	Loading
API success + valid payload	Result
WineApiException	Camera + Snackbar «Сервер недоступен...»
Другой Throwable	Camera + Snackbar «Не удалось распознать этикетку...»
Ошибка camera capture	Snackbar cameraError

Важно различать HTTP-ошибку и ошибку бизнес-разбора. WineApiService бросает WineApiException для non-2xx и для отсутствующего URL/некорректного envelope. ScanViewModel затем преобразует WineApiException в сообщение о недоступности сервера, а прочие ошибки — в сообщение о нераспознанной этикетке.

Loading

Состояние Loading моделируется явно через ScanUiState. Это предпочтительнее отдельного boolean, потому что state machine ограничена тремя состояниями: Camera, Loading, Result. Ошибка возвращает UI в Camera с errorMessage, которое отображается через SnackbarHost.

Детали вина

WineDetailScreenModel не показывает ошибку refresh пользователю, если локальная карточка уже есть. Это осознанный cache-first подход. Если локальной записи нет, notFound устанавливается после попытки чтения из БД.

14. Сборка, конфигурация и окружения

Сборка Android настраивается стандартными параметрами Gradle KMP-проекта (раздел 4): compileSdk/minSdk/targetSdk, JVM target 17. iOS-таргеты ограничены iosArm64 и iosSimulatorArm64, iosX64 намеренно исключён — сборка под классический Intel-симулятор не поддерживается. Версии всех ключевых зависимостей централизованы в libs.versions.toml (раздел 3), что упрощает единое обновление Kotlin/Compose/Ktor во всех модулях.

15. Тестирование и качество

В предоставленном архиве проекта отдельный тестовый модуль (unit/UI-тесты) не описан в документации — оценка тестового покрытия не входит в рамки данного документа и требует отдельного аудита кодовой базы. Рекомендуется в первую очередь

покрыть тестами доменный слой (domain/, mapping SQL ↔ domain ↔ network) и ScanViewModel, как наиболее критичную для пользовательского сценария часть.

16. Технический долг и рекомендации

- **Формализовать DI.** AppDependencies/LocalAppDependencies — рабочий, но ручной DI-механизм; по мере роста графа зависимостей стоит перейти на выделенный DI-фреймворк (Koin и т. п.).
 - **Асимметрия Android/iOS по ML.** Android имеет рабочий ONNX-адаптер (extractLabelCrop, хотя и не используемый текущим scan flow), iOS — только геометрический fallback-crop и TODO на подключение onnxruntime-objc. Стоит либо довести iOS до паритета, либо явно задокументировать, что on-device ML вообще не является частью текущего сценария (используется cropToGuideFrame).
 - **Неиспользуемый ONNX path на Android.** extractLabelCrop() реализован, но ScanRepository его не вызывает — либо включить в сценарий, либо удалить как мёртвый код, чтобы не вводить разработчиков в заблуждение.
 - **AvatarPicker.pickAvatar() на iOS не реализован** (возвращает null) — блокирует функциональность смены аватара на этой платформе.
 - **Унифицировать обработку ошибок.** Разделение WineApiException vs Throwable работает, но сообщения пользователю жёстко защиты в местах вызова — стоит централизовать маппинг ошибок в доменный слой.
 - **Debug-режим по умолчанию.** Аналогично backend (сохранение кропов), стоит убедиться, что DEBUG_SAVE_CROPS-подобные флаги на Android отключены в релизной сборке.
-

17. Onboarding нового frontend-разработчика

1. Изучить App.kt как composition root и понять, как создаются AppDependencies (раздел 5).
 2. Пройти сценарий сканирования по шагам (раздел 7) с открытым ScanViewModel и ScanRepository.
 3. Изучить границу слоёв SavedWine / SavedWineEntity / WineResult — это ключевой mapping во всём приложении.
 4. Собрать и запустить Android-таргет, затем — iOS-таргет (iosArm64/iosSimulatorArm64), сравнить платформенные различия из раздела 12.
 5. Перед внесением изменений — свериться с разделом 16 (технический долг), особенно по части DI и асимметрии ML Android/iOS.
-

18. Справочник файлов

Файл	Ключевая роль
LabelExtractor.ios.kt	iOS crop implementation / ONNX TODO
CameraController.ios.kt	AVFoundation implementation
NetworkModule.kt	Ktor configuration and timeout
AppDependencies.kt	Manual dependency container

Глоссарий

Термин	Значение
KMP	Kotlin Multiplatform — общий код для нескольких платформ
Compose Multiplatform	Общий декларативный UI на Compose
Voyager	Navigation + ScreenModel library
DTO	Data Transfer Object — модель сетевого JSON
Domain model	Модель, независимая от транспорта/БД
SQLDelight	Типобезопасный слой над SQLite
Flow / StateFlow	Реактивная передача состояния
expect / actual	KMP-механизм платформенных реализаций
ONNX Runtime	Runtime для выполнения ONNX-моделей
cache-first	Сначала показывается локальный кэш, затем выполняется refresh

Предоставленный frontend представляет собой компактное KMP-приложение с общим UI и domain/network/persistence слоями. Критический пользовательский путь — Camera → crop → POST /api/recognize → WineResult → SQLite → ResultScreen — реализован последовательно. Наиболее существенные точки развития: тестируемость, release-конфигурация, миграции БД, image cache и выравнивание iOS/Android возможностей.